

that conducting the same experiment again may yield slightly different numbers):

```
www.flipkart.com: 4
rukminim1.flixcart.com : 40
img1a.flixcart.com: 18
1.rome.api.flipkart.com : 16
rome.api.flipkart.com: 3
flipkart.d1.sc.omtrdc.net: 2
desa.fkapi.net : 1
desa.fkapi.net : 1
desr.fkapi.net : 1
dpm.demdex.net : 1

Num hosts = 10
Num txns = 87
Total content = 731550 bytes in 65
non-zero responses
```

To repeat, just to render the front page of Flipkart, the browser had to make connections with 10 different hosts and triggered 87 HTTP requests, with a maximum of 40 requests to one of the hosts. I also observed that the total content downloaded was approximately 714kB, just to display this page.

I tabulated similar data for the front pages of a few more popular portals to see if this trend remains the same (see Table 1).

While this is just data from a few portals, it does show that each modern portal triggers multiple connections and HTTP transactions. In fact, the news site is seen to be the most 'chatty', followed by the shopping site. While this data set is tiny, it does depict the trend. The current averages of the number of connections and the number of requests per page are 14 and 74, respectively (see <https://httparchive.org/reports/state-of-the-web>). Let us also be reminded that this is only with the front page of the portal – as one navigates across the portal, more contents will be available, requiring more connections and HTTPS requests/responses.

Table 1

Portal	Number of hosts	Number of transactions	Total data (kB)
<i>www.facebook.com</i>	6	59	1913
<i>www.linkedin.com</i>	12	39	147
<i>www.ndtv.com</i>	100	599	5525
<i>www.amazon.in</i>	33	337	2038

The reader will notice that with so many transactions, the limited concurrency of HTTP/1.1 can be a big challenge in rendering the page quickly. While the browser may not have to wait for every transaction to complete, it may have to wait for data from some of the connections before it can start rendering the page. Typically, it needs to wait for the CSS and JavaScripts to be available before it can render the page. Let us follow one example in detail to get a feel of the sequence. Here we enumerate the step performed by the client (i.e., the browser) and the minimum time taken for that step. To keep things general, it is common to measure time in units of RTT or round trip time, i.e., the time taken for a packet to travel from the client to the server and back.

With an RTT of 25ms, it takes 175ms before the browser completes Step 5 in the table given at the bottom of this page. While this may seem like a very insignificant time duration, research indicates that users are most engaged when the lag is within the range of 500ms; anything greater than 1000ms can cause users to switch their attention to other pages. Of course, this data may be from western consumers and Indians may be more patient, but

the RTT from India may also be higher.

As seen here, even after the connection and TLS setup, it takes another 3 RTT to receive the key contents even if the server was ready to serve them all in Step 3. This implies that even if the server had the processing capacity to serve such contents simultaneously, because of protocol limitations, it would still have to process requests in the order they arrived, leading to the additional latency.

Similarly, the TCP and TLS layers also add a minimum of 5 RTT for every connection.

A look at how protocols evolved

Before we get to the actual protocols, let's look at the new protocols that have been either recently specified or are in the process of being defined. Let us also examine which part of the stack they operate in, bringing in performance improvements.

1. Transport layer: TCP to QUIC
2. Session layer: TLS 1.2 to TLS 1.3
3. Application layer (or layer 7): HTTP 1.1 to HTTP/2 to HTTP/3

In this article, we will be focusing on the design of the HTTP/2 protocol and the performance improvements that it is expected to bring in. We also

1	Client opens TCP connection to server	2 RTT
2	Client performs TLS handshake with server	3 RTT
3	Client sends GET request for HTML and receives the HTML	1 RTT
4	Client sends GET request for CSS and receives the CSS	1 RTT
5	Client sends GET request for JS and receives the JS	1 RTT
6	Client sends 'N' other GET requests and receives the contents	N RTT